

TERRAFORM 2

task document

1) Watch the terraform-02 video.

Topics in video-2

Mutable vs Immutable Infrastructure :

1. Mutable Infrastructure

- **Definition:** Existing infrastructure is updated/modified to reach the new desired state.
- Example: You update a running EC2 instance to install a new version of software.
- **Problem:** Over time, changes may cause **configuration drift** (system differs from code definition).

2. Immutable Infrastructure

- **Definition:** Instead of modifying existing infrastructure, Terraform destroys the old resource and **creates a fresh one** with the new configuration.
- Example: Replace an EC2 instance with a new one that has the updated software.
- **Advantage:** Ensures consistency and avoids drift.
- **Trade-off:** May cause downtime if not carefully managed.

Terraform Lifecycle Rules

Terraform allows you to control how resources are created, destroyed, or modified with the lifecycle block.

Variables in Terraform

Variables make Terraform configurations **dynamic and reusable**.

Basic Structure:

```
variable "filename" {  
  default = "/root/pets.txt"  
  type    = string  
  description = "Path to the pets file"  
}
```

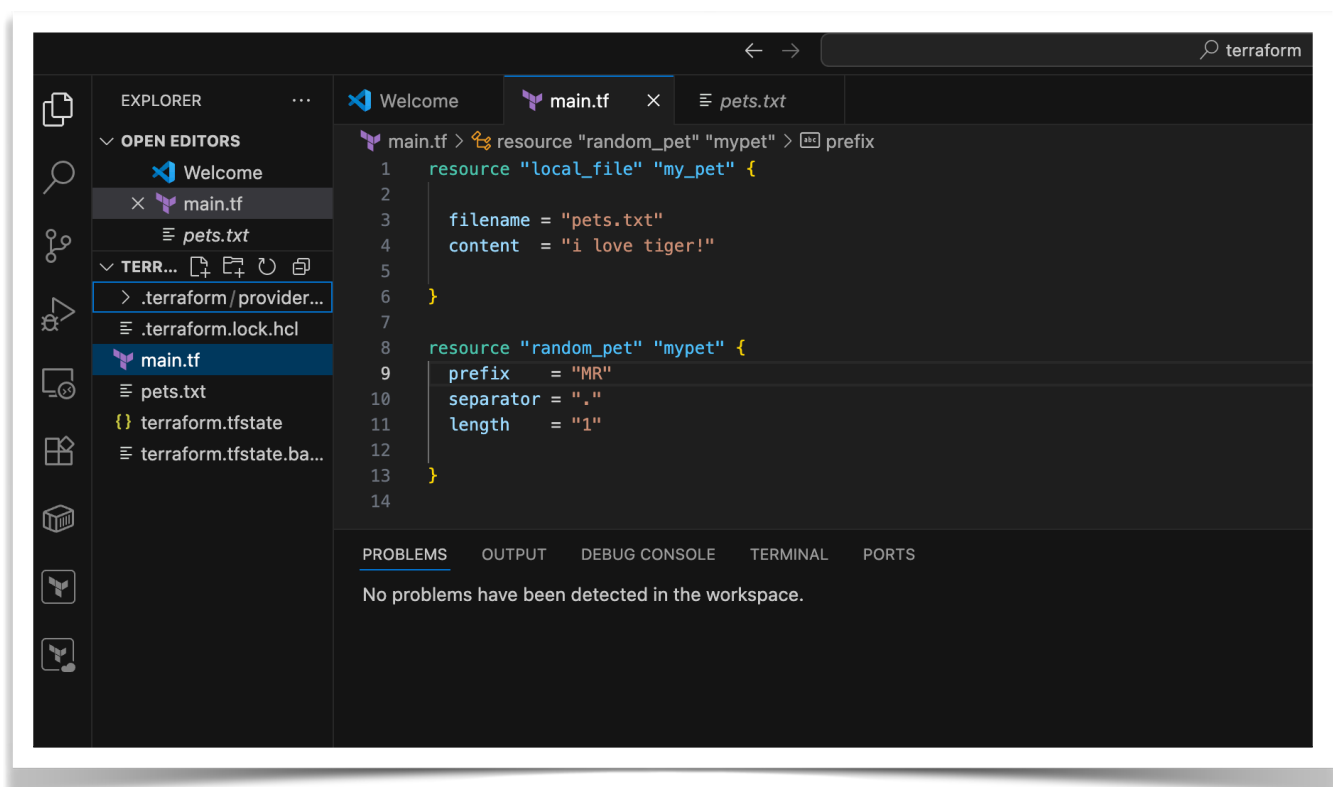
- **default** → Optional default value.
- **type** → Ensures variable type safety.
- **description** → Documentation for humans.

Variable Types

Type	Example	Use Case
string	"I love pets"	Text values
number	1	Numeric values (CPU, port numbers)
bool	true / false	Flags, toggles
any	Accepts any type	Flexible use
list	["cat", "dog"]	Ordered collection of items
map	{ pet1 = "cat", pet2 = "dog" }	Key-value pairs
object	{ name = "dog", age = 5 }	Complex structured data
tuple	["dog", 5, true]	Mixed data types in fixed order

2) Execute all the templates shown in video.

Main.tf -- main configuration file containing resource definition

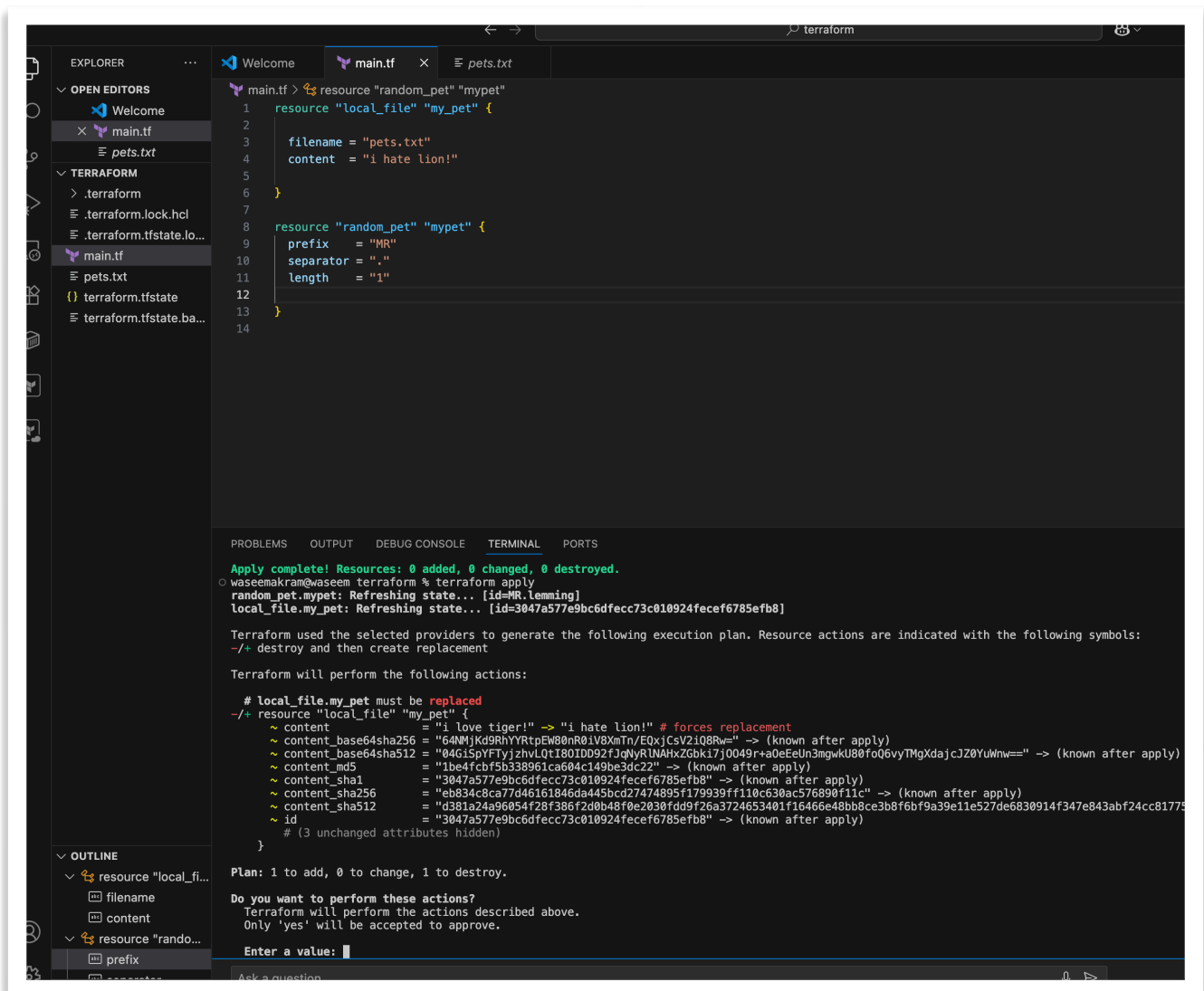


Terraform mutable vs immutable infrastructure:

Terraform as a IAC tool uses immutable infrastructure strategy.

Immutable means deleting the older infra and creating a newer one with a new update.

Mutable means using the existing infra and updating the system with newer versions



The screenshot shows a VS Code editor with the following components:

- EXPLORER:** Shows the file structure with folders for `.terraform` and `main.tf`. The `main.tf` file is selected.
- MAIN EDITOR:** Displays the Terraform configuration for `main.tf`. It contains two resources:

```
1 resource "random_pet" "mypet" {
2
3   filename = "pets.txt"
4   content  = "i hate lion!"
5 }
6
7
8 resource "random_pet" "mypet" {
9   prefix    = "MR"
10  separator = ","
11  length    = "1"
12 }
13
14
```
- TERMINAL:** Shows the output of the `terraform apply` command. It indicates that the resources were successfully applied and provides a detailed execution plan.

```
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
waseemakram@waseem terraform % terraform apply
random_pet.mypet: Refreshing state... [id=MR.lemming]
local_file.my_pet: Refreshing state... [id=3047a577e9bc6dfec73c010924fecef6785efb8]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

# local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
  ~ content      = "i love tiger!" -> "i hate lion!" # forces replacement
  ~ content_base64sha256 = "64NMjKd9RHYRtpEW80nR0iV8XmTn/E0xjCsV2iQ8Rw=" -> (known after apply)
  ~ content_base64sha512 = "04G1SpYFTyjzhvLQtI80IDD92fJqNyRlNAHxZGbk17j0049r+a0eEeUn3mgwkU80foQ6vyTMgXdajcJZ0YuWnw==" -> (known after apply)
  ~ content_md5      = "1be4fcfb5b338961ca604c149be3dc22" -> (known after apply)
  ~ content_sha1     = "3047a577e9bc6dfec73c010924fecef6785efb8" -> (known after apply)
  ~ content_sha256   = "eb834c8ca77d46161846da445bcd27474895f179939ff110c630ac576890f11c" -> (known after apply)
  ~ content_sha512   = "d381a24a96054f28f386f2d0b48f0e2030fdd9f26a3724653401f16466e48bb8ce3b8f6bf9a39e11e527de6830914f347e843abf24cc81775" -> (known after apply)
  ~ id              = "3047a577e9bc6dfec73c010924fecef6785efb8" -> (known after apply)
  # (3 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: 
```

Terraform will perform the following actions:

```
# local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
  ~ content      = "i love tiger!" -> "i hate lion!" # forces replacement
  ~ content_base64sha256 = "64NMjKd9RhYYRtpEW80nR0iV8XmTn/EQxjCsV2iQ8Rw=" -> (known after apply)
  ~ content_base64sha512 = "04GiSpYFTyjhvLQtI80IDD92fJqNyRLNAHxZGbki7j0049r+a0eEeUn3mgwkU80foQ6vyT" -> (known after apply)
  ~ content_md5      = "1be4fcfb5b338961ca604c149be3dc22" -> (known after apply)
  ~ content_sha1      = "3047a577e9bc6dfecc73c010924fecef6785efb8" -> (known after apply)
  ~ content_sha256     = "eb834c8ca77d46161846da445bcd27474895f179939ff110c630ac576890f11c" -> (known after apply)
  ~ content_sha512     = "d381a24a96054f28f386f2d0b48f0e2030fdd9f26a3724653401f16466e48bb8ce3b8f6" -> (known after apply)
  ~ id               = "3047a577e9bc6dfecc73c010924fecef6785efb8" -> (known after apply)
  # (3 unchanged attributes hidden)
}
```

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

```
local_file.my_pet: Destroying... [id=3047a577e9bc6dfecc73c010924fecef6785efb8]
local_file.my_pet: Destruction complete after 0s
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=d2c882e1898fcb2b75631dbfbc8ee75ad2fbb781]
```

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

○ waseemakram@waseem terraform %

Ask a question

Lifecycle :

lifecycle is a special block inside a resource that tells Terraform how to behave when a resource is created, updated, or deleted.

—> In simple words:

Lifecycle controls what Terraform should do and what it should NOT do with a resource.

1. Create_before_destroy :

Terraform creates a **new resource first**, then deletes the **old one**.

```
1 resource "local_file" "my_pet" {
2
3     filename = "pets.txt"
4     content  = "i hate lion!"
5
6     lifecycle {
7         create_before_destroy = true
8     }
9 }
10
11
12
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
local_file.my_pet: Refreshing state... [id=d2c882e1898fcb2b75631dbfbc8ee75ad2fbb781]
random_pet.mypet: Refreshing state... [id=MR.Lemming]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
- destroy

Terraform will perform the following actions:

# random_pet.mypet will be destroyed
# (because random_pet.mypet is not in configuration)
- resource "random_pet" "mypet" {
  - id      = "MR.Lemming" -> null
  - length  = 1 -> null
  - prefix  = "MR" -> null
  - separator = "." -> null
}

Plan: 0 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

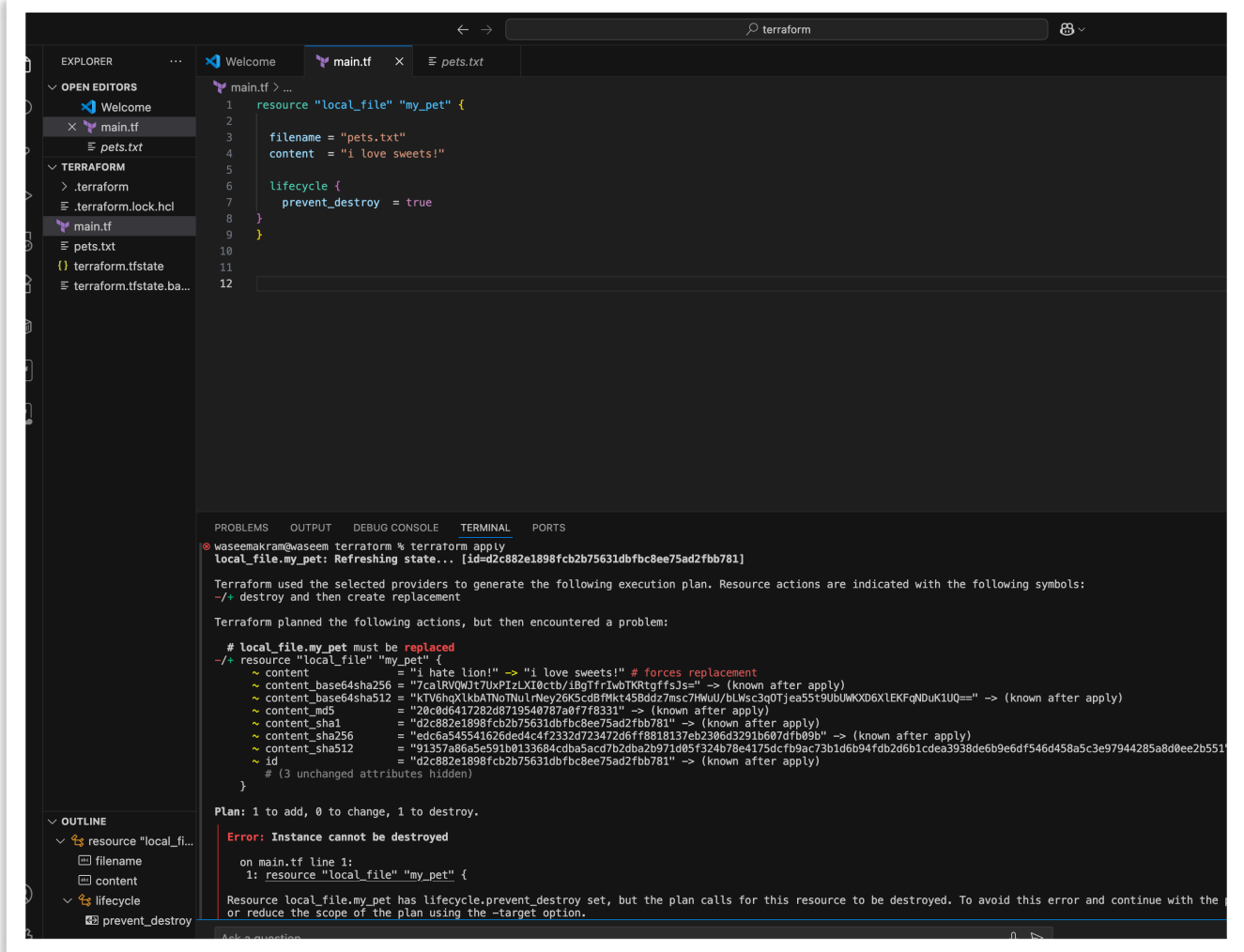
random_pet.mypet: Destroying... [id=MR.Lemming]
random_pet.mypet: Destruction complete after 0s

Apply complete! Resources: 0 added, 0 changed, 1 destroyed.
waseemakram@waseem terraform %
```

After adding the `create_before_destroy` inside lifecycle
—> save and terraform apply inside terminal

2. Prevent_destroy

- Prevents accidental deletion of critical resources (e.g., DB, VPC).
- Terraform will throw an error if terraform destroy tries to delete it.

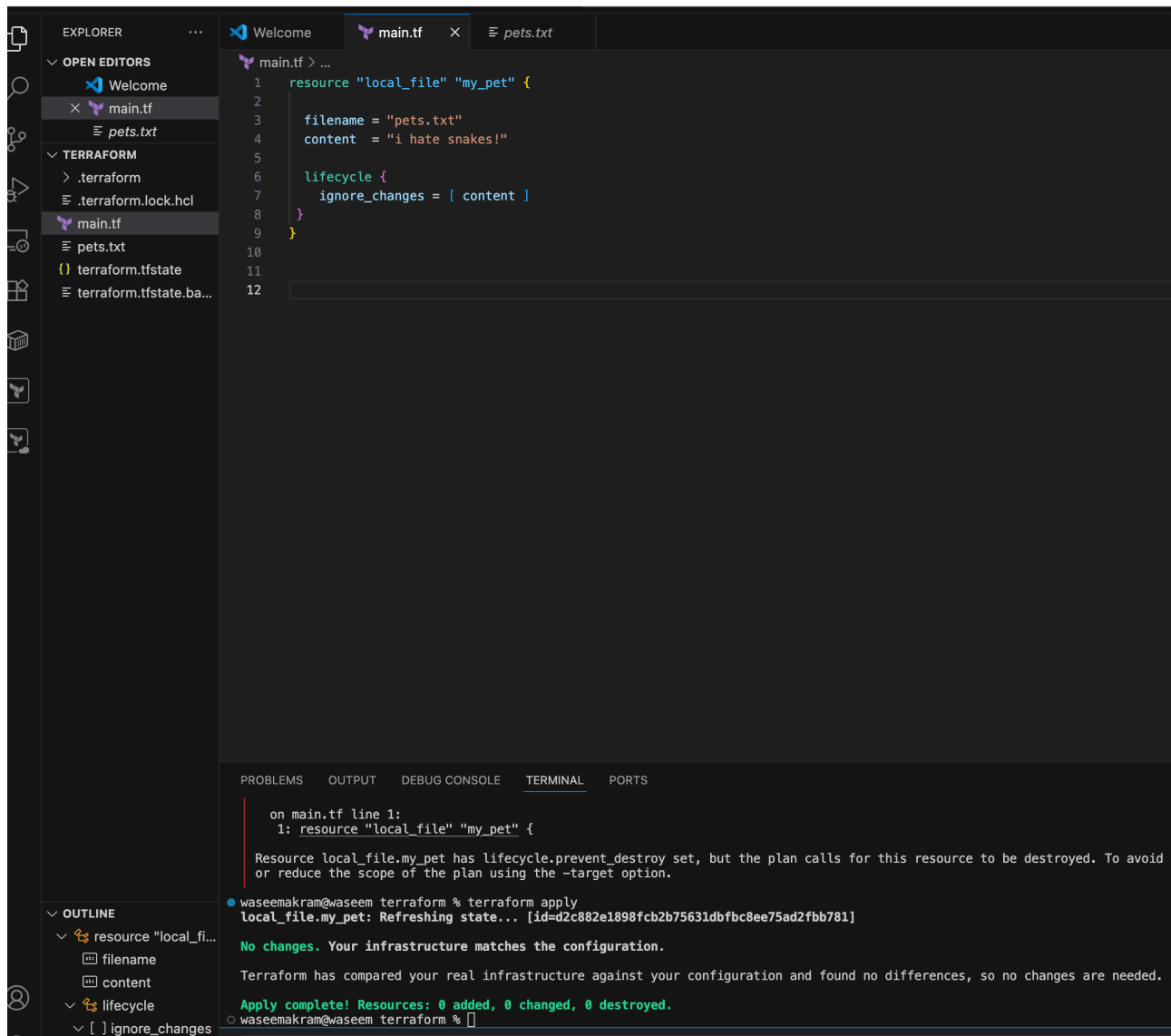


—> change some content inside and add prevent_destroy inside lifecycle

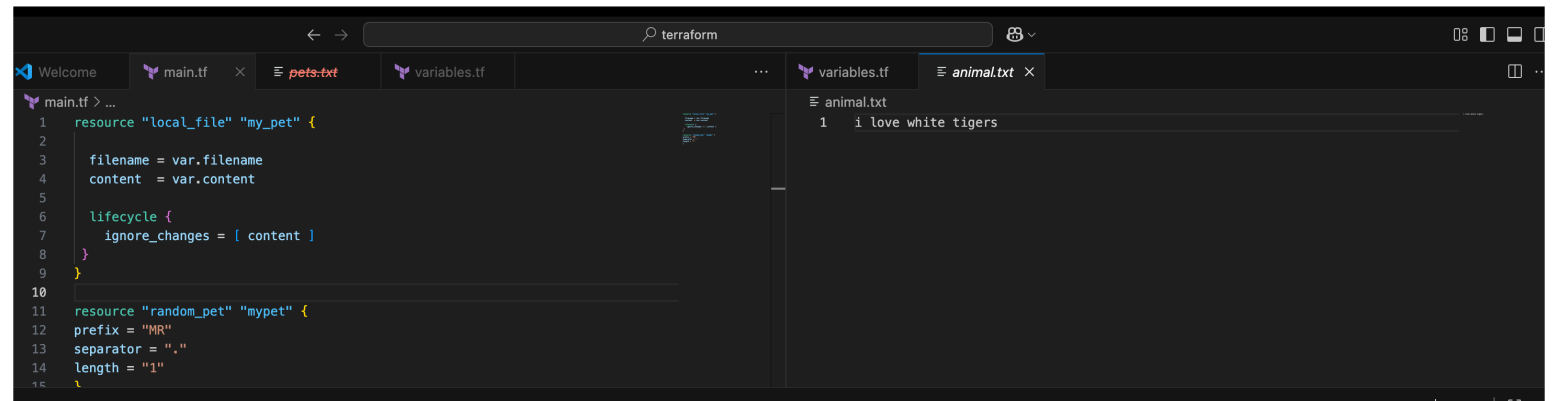
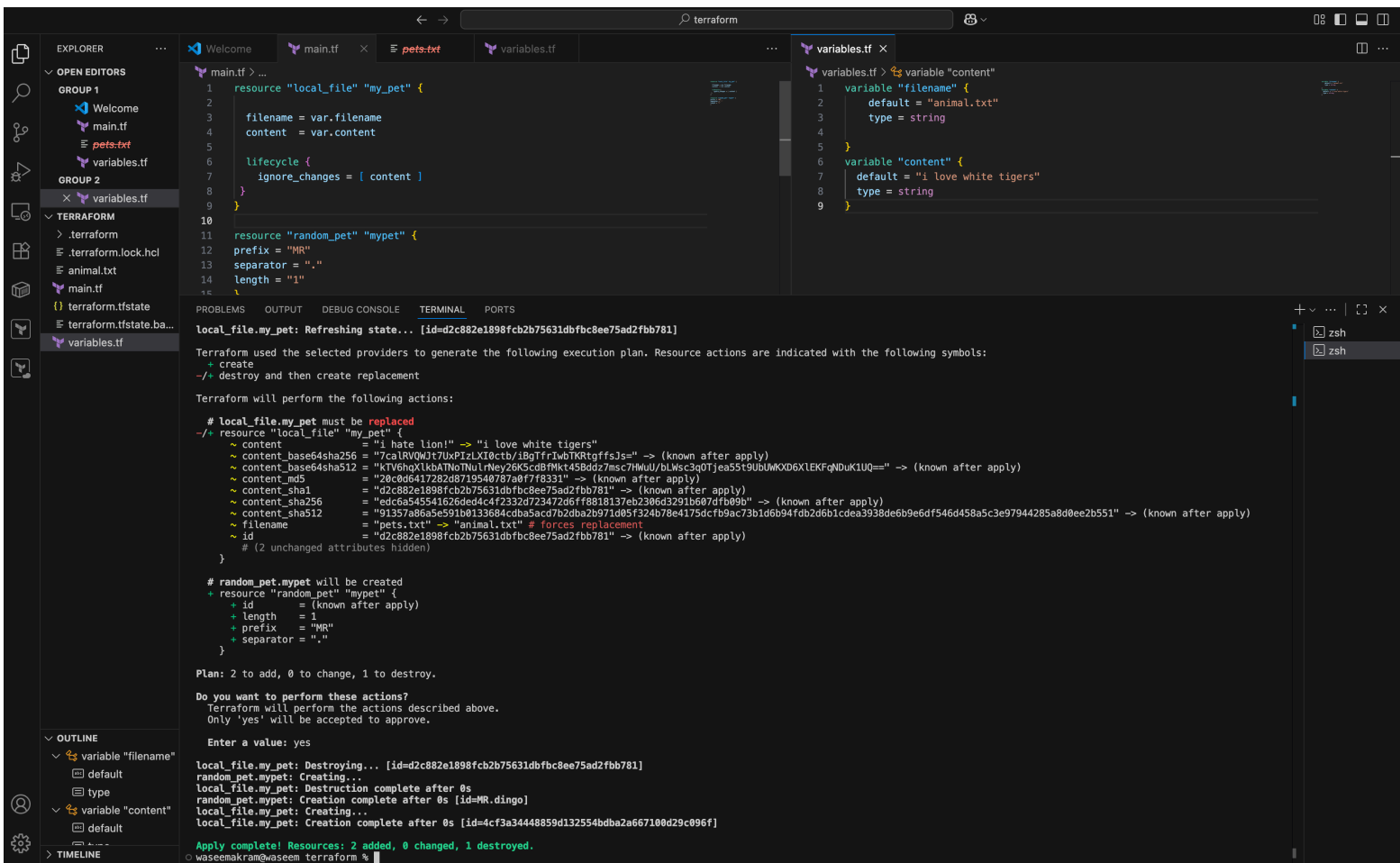
—> and save and terraform apply

3. Ignore_changes

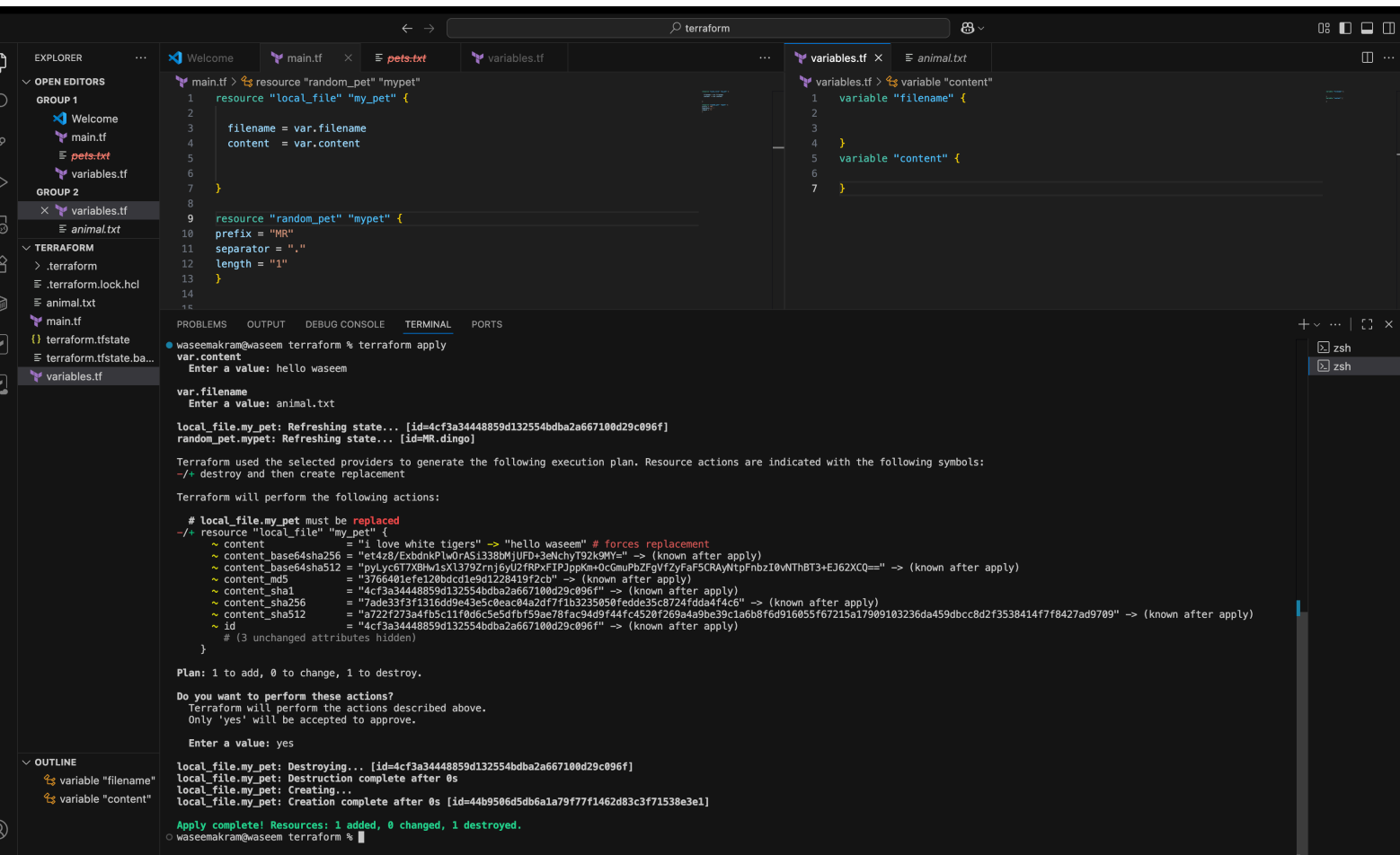
- Ignores specific attributes from being managed by Terraform.
- Useful when certain values are changed **outside Terraform** (e.g., tags added by another tool).
- Example: Ignore tags or content.



Variables in Terraform



Passing values using command



The screenshot shows a VS Code editor with several files open: `main.tf`, `pets.txt`, `variables.tf`, and `animal.txt`. The `main.tf` file contains Terraform configuration for a random pet resource. The `variables.tf` file defines variables for filename and content. The terminal shows the execution of `terraform apply` with interactive prompts for `var.content` and `var.filename`. The output displays the state refresh, the execution plan, and the actions Terraform will perform, including replacing the `local_file.my_pet` resource.

```
main.tf > resource "random_pet" "mypet"
1 resource "local_file" "my_pet" {
2   filename = var.filename
3   content  = var.content
4 }
5
6
7
8
9 resource "random_pet" "mypet" {
10   prefix = "MR"
11   separator = ","
12   length = "1"
13 }
14
15

variables.tf > variable "content"
1 variable "filename" {
2
3
4 }
5 variable "content" {
6
7 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
waseemakram@waseem terraform % terraform apply
var.content
Enter a value: hello waseem

var.filename
Enter a value: animal.txt

local_file.my_pet: Refreshing state... [id=4cf3a34448859d132554bdba2a667100d29c096f]
random_pet.mypet: Refreshing state... [id=MR.dingo]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

# local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
  ~ content          = "i love white tigers" -> "hello waseem" # forces replacement
  ~ content_base64sha256 = "et428/ExbdnkPwDrASi338bMjUFD+3eNchyT92k9MY=" -> (known after apply)
  ~ content_base64sha512 = "pyLyc6T7XBHw1sXl379Zrnj6yU2FRPxFlPppKm+0cGmuPbZFGvFzYFaF5CRAyNtpFnbzI0WThBT3+EJ62XCQ==" -> (known after apply)
  ~ content_md5        = "3766401ef120bdc1e9d1228419f2cb" -> (known after apply)
  ~ content_sha1       = "4cf3a34448859d132554bdba2a667100d29c096f" -> (known after apply)
  ~ content_sha256     = "7ade33f3f1316d9e43e5c9eac04a2d77f1b3235050fedde35c8724fdda4f4c6" -> (known after apply)
  ~ content_sha512     = "a722f273a4fb5c11f0d6c5e5dfbf59ae78fac94d9f44fc4520f269a4a9be39c1a6b8f6d916055f67215a17909103236da459dbcc8d2f35384147f8427ad9709" -> (known after apply)
  ~ id                = "4cf3a34448859d132554bdba2a667100d29c096f" -> (known after apply)
  # (3 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.my_pet: Destroying... [id=4cf3a34448859d132554bdba2a667100d29c096f]
local_file.my_pet: Destruction complete after 0s
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=44b9506d5db6a1a79f77f1462d83c3f71538e3e1]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
waseemakram@waseem terraform %
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

waseemakram@waseem terraform % terraform apply
var.content
Enter a value: hello waseem

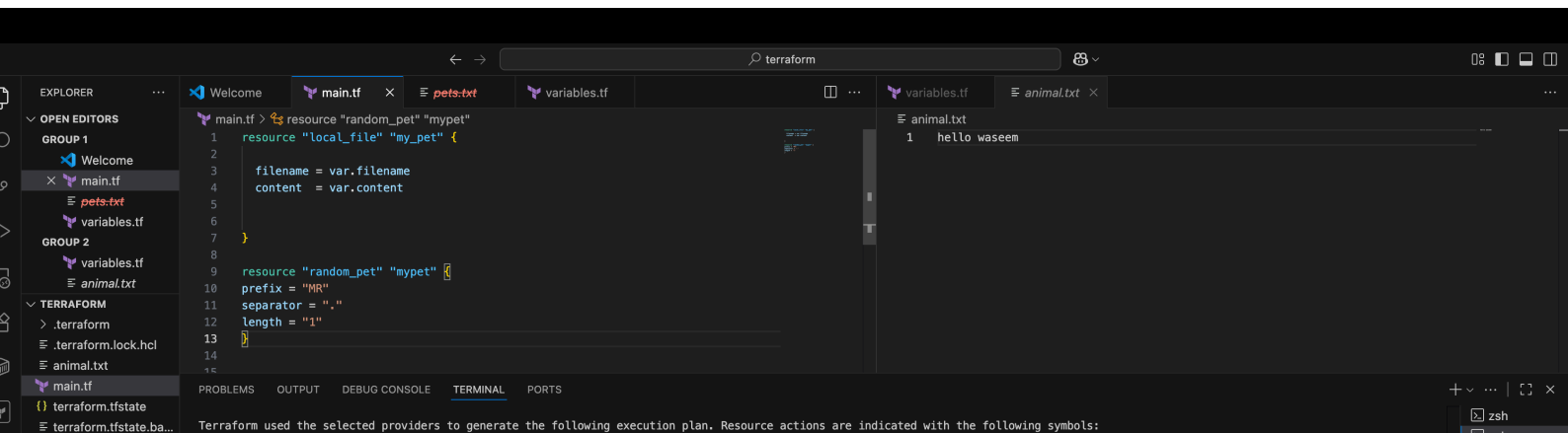
var.filename
Enter a value: animal.txt

local_file.my_pet: Refreshing state... [id=4cf3a34448859d132554bdba2a667100d29c096f]
random_pet.mypet: Refreshing state... [id=MR.dingo]

Terraform used the selected providers to generate the following execution plan. Resource ac
-/+ destroy and then create replacement

Terraform will perform the following actions:

# local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
  ~ content          = "i love white tigers" -> "hello waseem" # forces replacement
```



```
waseemakram@waseem terraform % terraform apply -var "filename=animal.txt" -var "content=hello this is waseem"
local_file.my_pet: Refreshing state... [id=44b9506d5db6a1a79f77f1462d83c3f71538e3e1]
random_pet.mypet: Refreshing state... [id=MR.dingo]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

# local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
  ~ content          = "hello waseem" -> "hello this is waseem" # forces replacement
  ~ content_base64sha256 = "CrKgpwXpu8Ku1NgxDXaLQkTaLb9FM+35REToqw/q2Q0=" -> (known after apply)
  ~ content_base64sha512 = "h/LaDXSTz0r9JPgFeJTCetTJAZ4RqQ08iuvv9CQU4HTs2qdw8MxZMYWYqD1H5mCnkIJ4A+Q2qW0mxNrSfp7w==" -> (known after apply)
  ~ content_md5         = "38c195bd0997fbf40671c275e2a5ce96" -> (known after apply)
  ~ content_sha1        = "44b9506d5db6a1a79f77f1462d83c3f71538e3e1" -> (known after apply)
  ~ content_sha256      = "0ab2a0a705e9b812aed4d8310d768b4244da2dbf4533edf94444e8ab0fead90d" -> (known after apply)
  ~ content_sha512      = "87f2da0d7e53ccea24f8057894c27ad4c9019e11a90048ba2befbfd0905381d3b36a9dc0133164c616caa0f51f99829e4209e00f90daac0dd26c4dad27e9" -> (known after apply)
  ~ id                 = "44b9506d5db6a1a79f77f1462d83c3f71538e3e1" -> (known after apply)
  # (3 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.my_pet: Destroying... [id=44b9506d5db6a1a79f77f1462d83c3f71538e3e1]
local_file.my_pet: Destruction complete after 0s
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=f89a3e3eb471b5aa10d86e5fdad317088b834faf]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
waseemakram@waseem terraform %
```

Terraform apply -var “filename=animal.txt” -var “content=hello this is waseem”

3) Integrate terraform in Jenkins using Terraform plugin.

Step 1: Install Terraform on Jenkins Server

On the Jenkins machine (EC2, VM, etc.):

Install Terraform

sudo yum install -y wget unzip # (Amazon Linux)

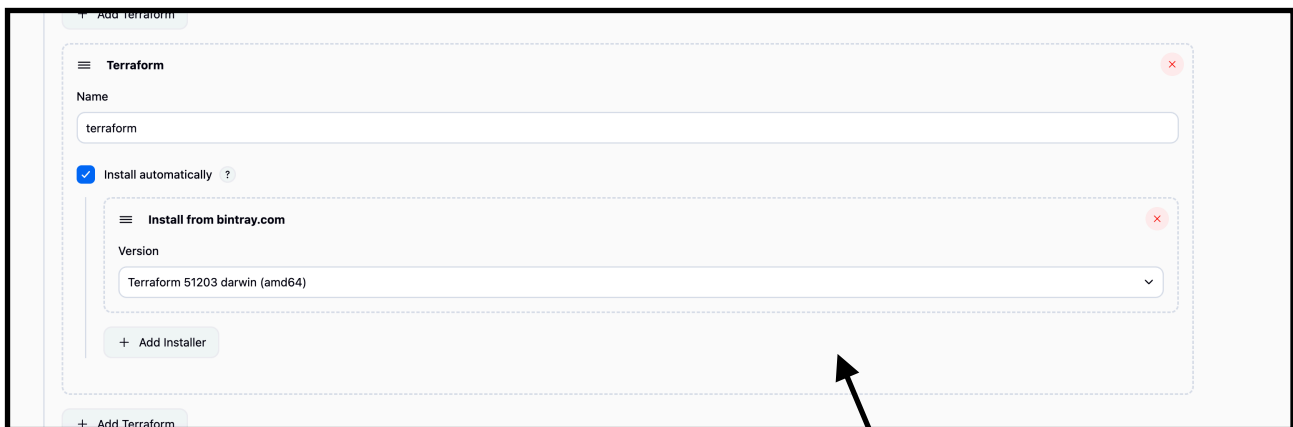
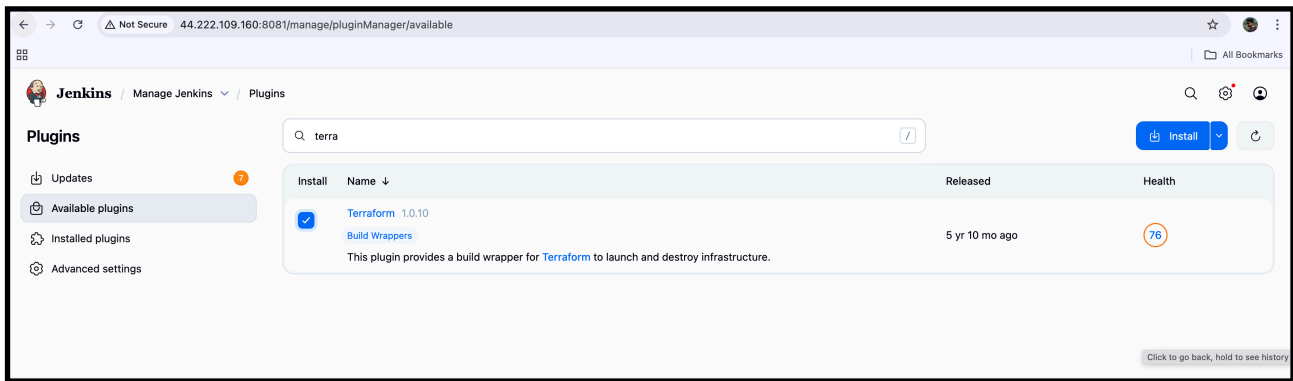
```
[root@ip-172-31-65-24 local]# cd bin
[root@ip-172-31-65-24 bin]# ls
terraform
[root@ip-172-31-65-24 bin]#
```

```
[root@ip-172-31-65-24 ~]# terraform -version
Terraform v1.14.3
on linux_amd64
```

```
[root@ip-172-31-65-24 ~]# ls
LICENSE.txt  backup.sh  jenkins-backups  jenkinsbackup  slave_workspace  terraform_1.14.3_linux_amd64.zip  test.pem
[root@ip-172-31-65-24 ~]#
```

Step 2: Install Terraform Plugin in Jenkins

1. Go to Jenkins Dashboard → Manage Jenkins → Plugins → Available Plugins.



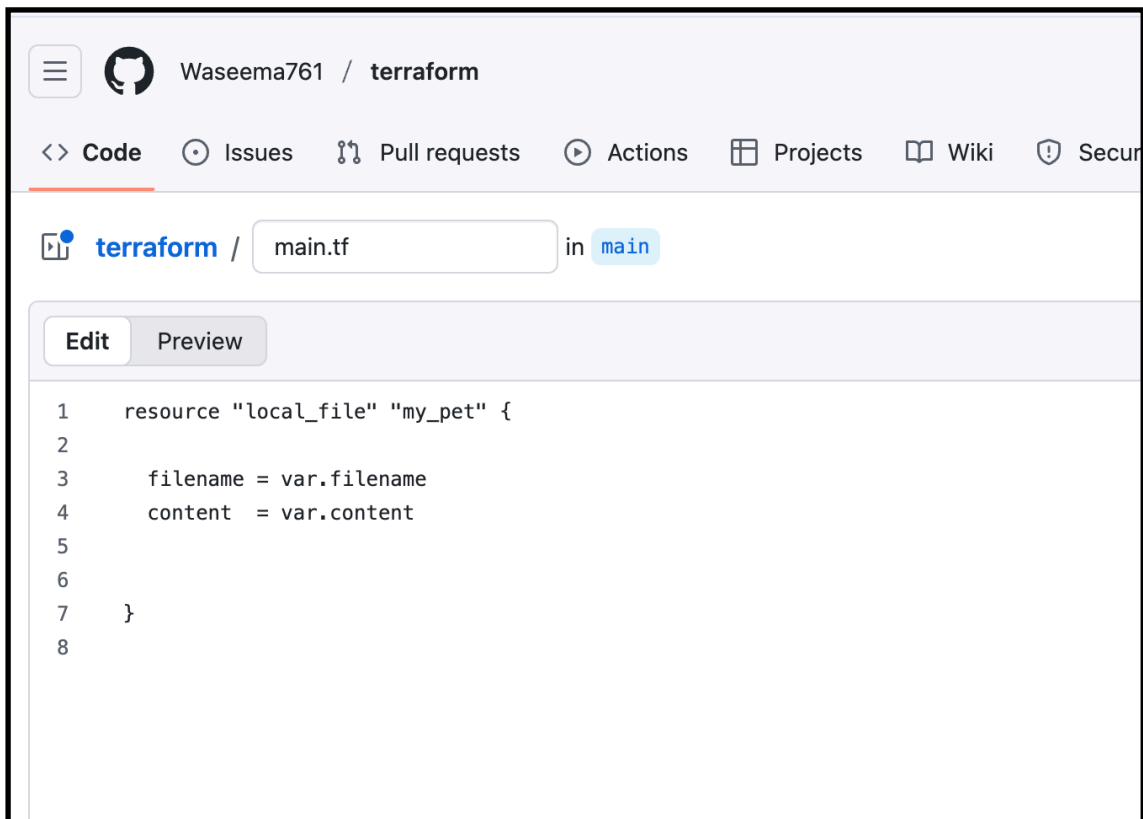
Step 3: Configure Terraform in Jenkins

1. Go to **Manage Jenkins** → **Global Tool Configuration**.
2. Find **Terraform** section.
3. Add a Terraform installation → give it a **Name** (example: Terraform-1.8.5) and path if manually installed.
 - If plugin supports auto-install → let Jenkins download it.
 - If already installed → point Jenkins to `/usr/local/bin/terraform`.

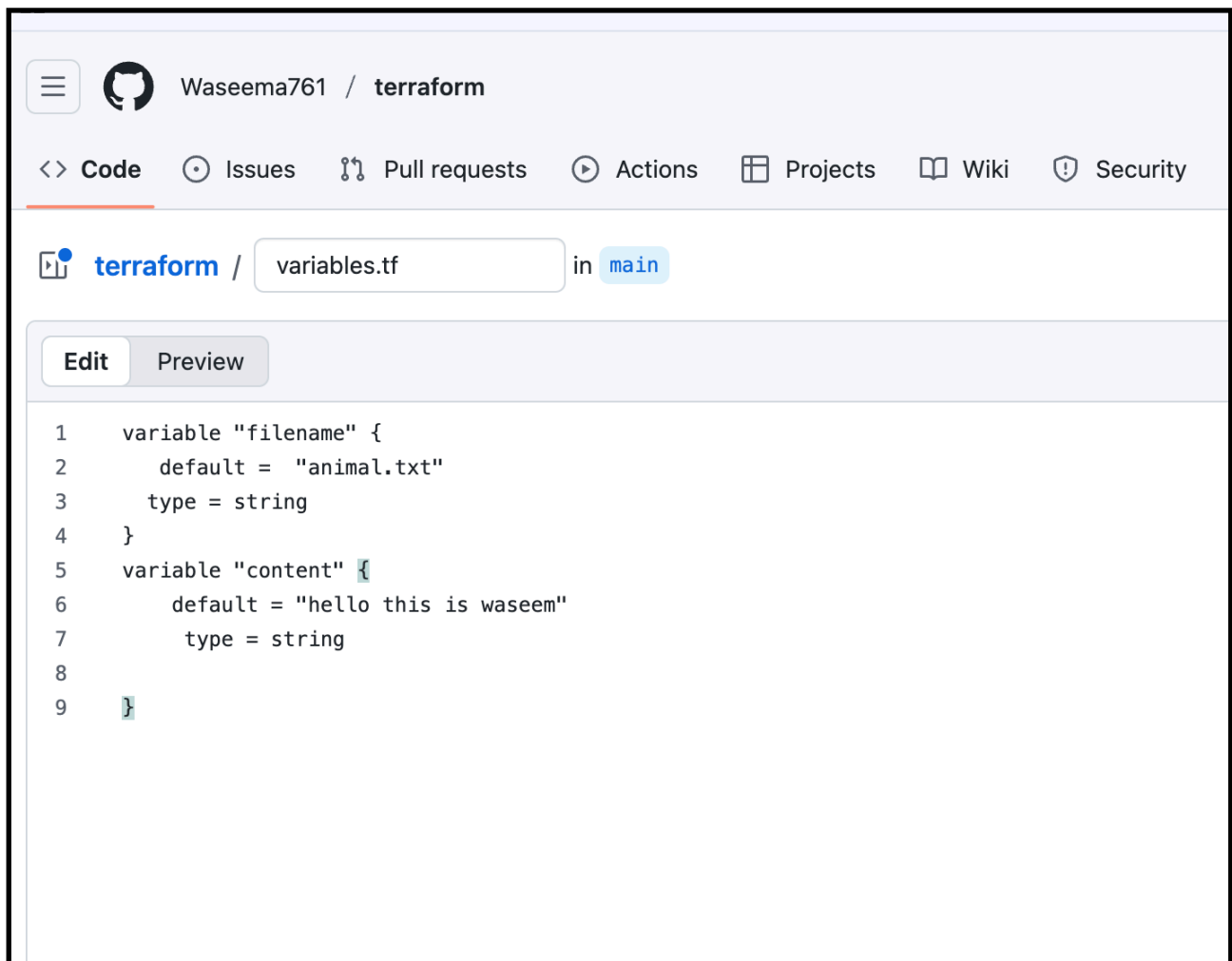
Step 4:

In Git hub add these files

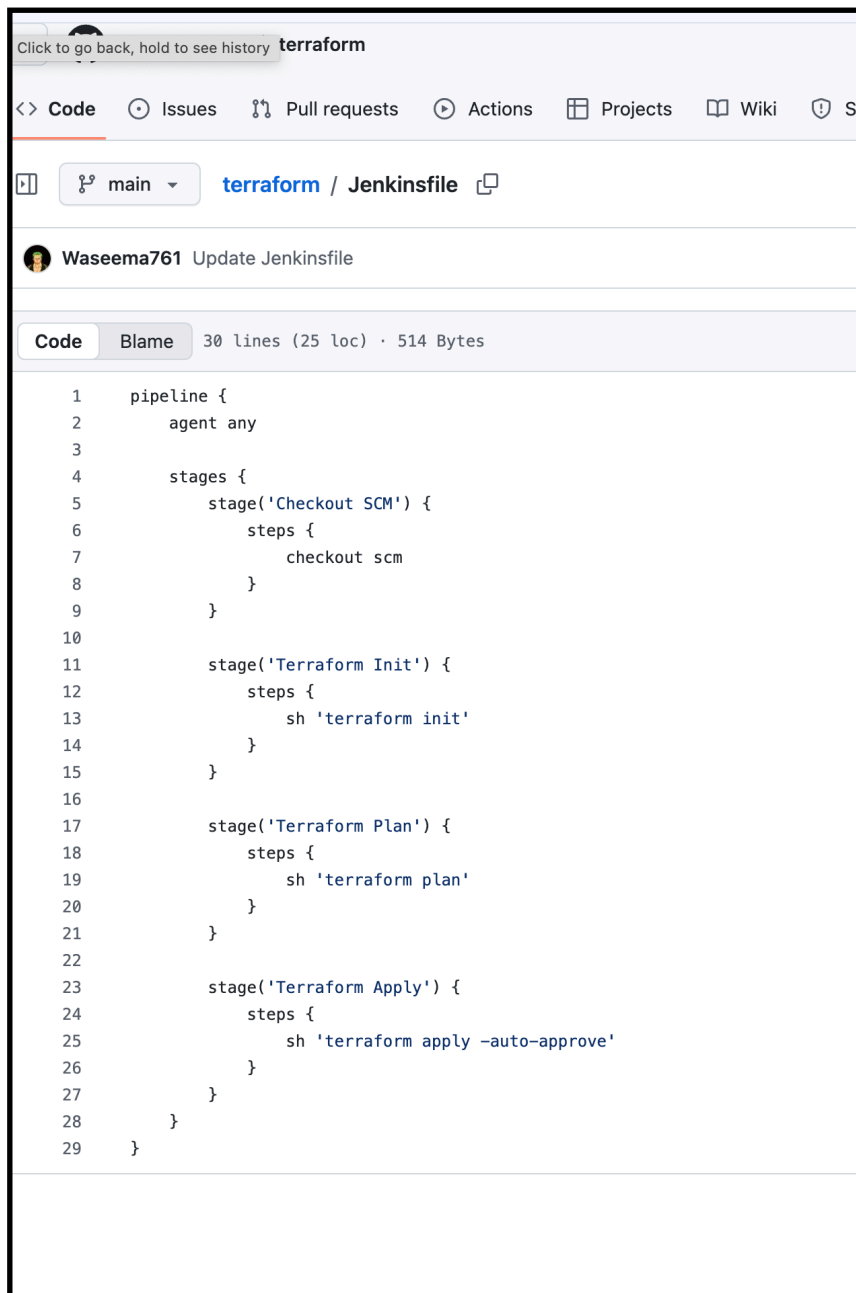
Main.tf



variables.tf



Jenkinsfile



```
1 pipeline {
2   agent any
3
4   stages {
5     stage('Checkout SCM') {
6       steps {
7         checkout scm
8       }
9     }
10
11    stage('Terraform Init') {
12      steps {
13        sh 'terraform init'
14      }
15    }
16
17    stage('Terraform Plan') {
18      steps {
19        sh 'terraform plan'
20      }
21    }
22
23    stage('Terraform Apply') {
24      steps {
25        sh 'terraform apply -auto-approve'
26      }
27    }
28  }
29 }
```

—-> now go to Jenkins gui and create a job with pipeline and add the scm ., add the the GitHub repo url and add the same branch name

Jenkins / terraform / Configuration

Configure

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/Waseema761/terraform.git

Credentials ?

- none -

+ Add

Advanced

+ Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/main

+ Add Branch

—> now save and build

← → ↻ ⚠ Not Secure 3.235.170.139:8081/job/terraform/

Jenkins / terraform

Status

</> Changes

▶ Build Now

⚙ Configure

🗑 Delete Pipeline

📁 Stages

✎ Rename

🔍 Pipeline Syntax

✅ terraform

Permalinks

- Last build (#5), 7 min 15 sec ago
- Last stable build (#5), 7 min 15 sec ago
- Last successful build (#5), 7 min 15 sec ago
- Last failed build (#3), 23 hr ago
- Last unsuccessful build (#3), 23 hr ago
- Last completed build (#5), 7 min 15 sec ago

Builds

Filter

Today

✅ #5 11:33 AM

Jenkins / terraform / #5 / Pipeline Overview

Manually run by waseem akram | Started 7 min 43 sec ago | Queued 13 ms | Took 5.1 sec | </> Changes

Graph

```

graph LR
    Start((Start)) --> CheckoutSCM1[Checkout SCM]
    CheckoutSCM1 --> CheckoutSCM2[Checkout SCM]
    CheckoutSCM2 --> TerraformInit[Terraform Init]
    TerraformInit --> TerraformPlan[Terraform Plan]
    TerraformPlan --> TerraformApply[Terraform Apply]
    TerraformApply --> End((End))
  
```

Search

- Checkout SCM 0.3s
- Checkout SCM 0.29s
- Terraform Init 0.32s
- Terraform Plan 0.56s
- Terraform Apply 0.31s**

Terraform Apply 0.31s Started 7m 42s ago

terraform apply -auto-approve

```

0 + terraform apply -auto-approve
1 local_file.my_pet: Refreshing state... [id=3047a577e9bc6dfecc73c010924fecef6785efb8]
2
3 Terraform used the selected providers to generate the following execution
4 plan. Resource actions are indicated with the following symbols:
5 + create
6
7 Terraform will perform the following actions:
8
9 # local_file.my_pet will be created
10 + resource "local_file" "my_pet" {
11   + content           = "hello this is waseem"
12   + content_base64sha256 = (known after apply)
13   + content_base64sha512 = (known after apply)
14   + content_md5        = (known after apply)
15   + content_sha1       = (known after apply)
  
```

Validation :

- > go to the Jenkins server and
- > cd /var/lib/jenkins/workspace/
- > cd terraform (job name)

```

[root@ip-172-31-65-24 ~]# cd /var/lib/jenkins/workspace/
[root@ip-172-31-65-24 workspace]# ls
'MULTI-STAGE PIPELINE'      hiring-app-declarative      job2      pipeline      'scripted pipeline'      simplecustomerapp-pipeline      vprofile
'MULTI-STAGE PIPELINE@tmp'  hiring-app-declarative@tmp  job3      pipeline@tmp  'scripted pipeline@tmp'  simplecustomerapp-pipeline@tmp
first-job                  hiring-app-parallel         parametrized-job      private      scripted-pipeline1      terraform
hiring-app                 hiring-app-parallel@tmp     parametrized-job@tmp  private@tmp  scripted-pipeline1@tmp  terraform@tmp
[root@ip-172-31-65-24 workspace]# cd terraform
[root@ip-172-31-65-24 terraform]# ls
Jenkinsfile  README.md  animal.txt  main.tf  pets.txt  terraform.tfstate  terraform.tfstate.backup  variables.tf
[root@ip-172-31-65-24 terraform]# cat animal.txt
hello this is waseem[root@ip-172-31-65-24 terraform]# cat animal.txt
hello this is waseem[root@ip-172-31-65-24 terraform]# cd ..
  
```

- > check ls we are able to see the content from the variable.tf file from GitHub

